

Solving the $3D\ N^2$ -Queens Problem with Monte-Carlo Markov Chain

Maxime Ducourau | 329544 | maxime.ducourau@epfl.ch

Jean Perbet | 341418 | jean.perbet@epfl.ch

Mehdi Zoghلامي | 326381 | mehdi.zoghلامي@epfl.ch

Code: <https://github.com/Mehdi-Zogh/mc-queens-problem>
totAI

Abstract

The $3D\ N^2$ -queens problem is a constraint satisfaction problem that asks us to place N^2 queens on an $N \times N \times N$ chessboard such that the least number of queens attack each other. The 3D variant generalizes the classical 2D N -queens problem by extending attack conditions to all 3D diagonals and planes. We solve this problem using Monte Carlo Markov Chain (MCMC) method with simulated annealing. Our approach treats the problem as an energy minimization task where the energy function counts the number of attacking queen pairs. We employ the Metropolis-Hastings algorithm with an exponential temperature schedule to explore the configuration space and guide the search toward low-energy solutions. We analyze the performance of our algorithm across different board sizes and characterize the scaling behavior of the minimal energy reached as a function of problem size. All code is available [here](#).

1 Introduction

The N -queens problem is a classical combinatorial optimization problem dating back to the 19th century. The problem asks to place N queens on an $N \times N$ chessboard such that no two queens can attack each other. In classical chess, a queen can attack any piece that shares its row, column, or diagonal. Solving this problem for small values of N is feasible via constraint propagation or backtracking algorithms, but as N grows, the search space becomes exponentially large.

The variant we tackle here extends this challenge to three dimensions. We place N^2 queens on an $N \times N \times N$ cubic chessboard where a queen at position (i, j, k) attacks positions sharing two coordinates (rows, columns and pillars) and all 2D and 3D diagonals passing through that position. This extension significantly increases the complexity of the constraint landscape, rendering traditional backtracking approaches inefficient.

Rather than using deterministic search methods, we employ Monte-Carlo Markov Chain (MCMC) method, a probabilistic sampling technique that has proven effective for constraint satisfaction and optimization problems. MCMC allows us to explore the high-dimensional configuration space in a statistically principled manner, balancing global exploration with local exploitation through simulated annealing.

The key contribution of this work is a modular, efficient MCMC implementation that:

1. Precisely formulates the $3D\ N^2$ -queens problem as an energy minimization task.
2. Implements the Metropolis-Hastings algorithm with $O(1)$ per-step complexity via efficient delta energy computation.
3. Employs exponential temperature annealing to escape local minima.
4. Characterizes the scaling behavior of solution quality with respect to board size.

2 Motivation: NP-hardness

The minimization of queen collisions on an $N \times N \times N$ board is NP-Hard.

- Relation to NP-Complete Problems: The decision problem (finding a zero-collision solution, $E(s) = 0$) is a variant of the generalized N -Queens problem, known to be NP-Complete. Consequently, our minimization task is NP-Hard.
- Combinatorial Explosion: The search space size, $\binom{N^3}{N^2}$, grows super-exponentially with N . This confirms the high complexity and the observed difficulty of finding optimal solutions for larger N within a fixed computational budget.

This complexity necessitates the use of stochastic search methods like the Metropolis algorithm.

3 Approach

3.1 Problem Formulation

3.1.1 State Space Representation

We represent a configuration as an $N^2 \times 3$ matrix of coordinates:

$$\mathbf{s} = \begin{bmatrix} i_1 & j_1 & k_1 \\ i_2 & j_2 & k_2 \\ \vdots & \vdots & \vdots \\ i_{N^2} & j_{N^2} & k_{N^2} \end{bmatrix} \quad (1)$$

Each row represents the position (i_m, j_m, k_m) of the m -th queen, where $i_m, j_m, k_m \in \{1, 2, \dots, N\}$. This representation is computationally efficient: moving a queen requires updating a single row, and checking attack conditions requires only coordinate comparisons.

3.1.2 Attack Conditions

Two queens at positions (i_1, j_1, k_1) and (i_2, j_2, k_2) attack each other in the following cases.

Rows, Columns & Pillars

Same plane + one common coordinate.

- $i_1 = i_2 \wedge j_1 = j_2 \wedge k_1 \neq k_2$
- $i_1 = i_2 \wedge k_1 = k_2 \wedge j_1 \neq j_2$
- $j_1 = j_2 \wedge k_1 = k_2 \wedge i_1 \neq i_2$

2D (Plane) Diagonals

- $|i_1 - i_2| = |j_1 - j_2| \wedge k_1 = k_2 \wedge |i_1 - i_2| \neq 0$
- $|i_1 - i_2| = |k_1 - k_2| \wedge j_1 = j_2 \wedge |i_1 - i_2| \neq 0$
- $|j_1 - j_2| = |k_1 - k_2| \wedge i_1 = i_2 \wedge |j_1 - j_2| \neq 0$

3D (Space) Diagonals

- $|i_1 - i_2| = |j_1 - j_2| = |k_1 - k_2| \wedge |i_1 - i_2| \neq 0$

3.1.3 Energy Function

We define the energy as the total number of "collisions" between queens, i.e. the number of attacking queen pairs.

$$E(\mathbf{s}) = \sum_{m=1}^{N^2} \sum_{n=m+1}^{N^2} \mathbb{1}[m \text{ and } n \text{ attack each other}] \quad (2)$$

where $\mathbb{1}[\cdot]$ is the indicator function. A valid solution satisfies $E(\mathbf{s}) = 0$. This formulation directly aligns the objective we wish to optimize (zero conflicts) with the energy we measure, creating a smooth landscape that guides optimization.

3.2 Energy Computation

Calculating the energy of a state can be computationally expensive if done naively. A brute-force approach would require checking all $O(N^4)$ possible queen pairs for conflicts, which is impractical for large values of N . To optimize this process, we compute the energy only once for the initial state. For subsequent states, we leverage the concept of delta energy: when a single queen is moved to a new, unoccupied position, we calculate the change in energy by subtracting the number of queens attacking the moved queen in its previous position to the number attacking it in its new position. This delta is then added to the current energy, yielding the energy of the new state and allowing $O(N^2)$ per-state computational complexity.

$$\Delta E = \sum_{n \neq m} \mathbb{1}(\text{attack}(\mathbf{p}'_m, \mathbf{p}_n)) - \mathbb{1}(\text{attack}(\mathbf{p}_m, \mathbf{p}_n)) \quad (3)$$

where $\mathbf{p}_m$ is the old position and $\mathbf{p}'_m$ is the proposed new position.

We can go even further by optimizing the way we compute how many queens attack a given position. Using hash table-based tracking structures that record how many queens occupy each plane xy , xz and yz , as well as each of the 6 face diagonals and 4 space diagonals, we can leverage constant-time lookups to precisely know the number of collisions a given queen is involved into. This brings our initial energy computation cost to $O(N^2)$, while delta energy computation cost becomes $O(1)$, making the algorithm feasible for large board sizes where brute-force methods would be computationally prohibitive.

3.3 Algorithm

3.3.1 Target Distribution

We define a probability distribution that favors low-energy configurations:

$$\pi(\mathbf{s}) = \frac{1}{Z} e^{-\beta E(\mathbf{s})} \quad (4)$$

where $Z = \sum_{\mathbf{s}'} e^{-\beta E(\mathbf{s}')}$ is the partition function and β is the inverse temperature. Configurations with lower energy have exponentially higher probability. The parameter β controls the selectivity: small β yields a nearly uniform distribution (exploration), while large β concentrates probability mass on low-energy states (exploitation).

3.3.2 Metropolis-Hastings

The MCMC chain maintains states $s_0, s_1, s_2, \dots$. At each iteration t :

1. **Proposal** Sample candidate s' by selecting a random queen uniformly and proposing a uniformly random unoccupied position on the board. Let $q(s'|s)$ denote this symmetric proposal distribution.
2. **Acceptance** Compute the acceptance probability:

$$\alpha(s \rightarrow s') = \min \left(1, \frac{\pi(s')q(s|s')}{\pi(s)q(s'|s)} \right) \quad (5)$$

Since our proposal is symmetric ($q(s|s') = q(s'|s)$), this simplifies to the following.

$$\alpha(s \rightarrow s') = \min \left(1, e^{-\beta(E(s') - E(s))} \right) \quad (6)$$

$$= \min(1, e^{-\beta \Delta E}) \quad (7)$$

Then, draw $u \sim \text{Uniform}(0, 1)$. If $u < \alpha$, accept the move: $s_{t+1} = s'$. Otherwise, $s_{t+1} = s_t$.

This criterion is derived from the detailed balance condition, ensuring that the stationary distribution of the chain is $\pi(s)$.

$$\pi(s)P(s \rightarrow s') = \pi(s')P(s' \rightarrow s) \quad (8)$$

3.3.3 Simulated Annealing

To balance exploration and exploitation, we employ simulated annealing. The inverse temperature follows an exponential schedule:

$$\beta(t) = \beta_0 \cdot c^t \quad (9)$$

where β_0 is the initial temperature (typically 0.1) and $c > 1$ (typically 1.001) is the cooling rate. Early in the run, small β allows the chain to explore broadly, helping escape local minima. As β increases, the chain becomes greedy, accepting only moves that decrease energy, thus intensifying the search in promising regions.

Algorithm: MCMC with Simulated Annealing

Input: Board size N , iteration budget T , cooling rate c , β_0

Initialization:

$s \leftarrow \text{RandomConfiguration}(N)$

$\beta \leftarrow \beta_0$

Main Loop:

for $t = 1$ **to** T **do**

$m \leftarrow \text{RandomInteger}(1, N^2)$

$p' \leftarrow \text{RandomUnoccupiedPosition}(N)$

$\Delta E \leftarrow \text{ComputeDeltaEnergy}(s, m, p')$

$u \leftarrow \text{Uniform}(0, 1)$

if $\log(u) < -\beta \cdot \Delta E$ **then**

$s[m] \leftarrow p'$

end if

$\beta \leftarrow \beta \cdot c$

end for

Return: s

3.4 Key Properties

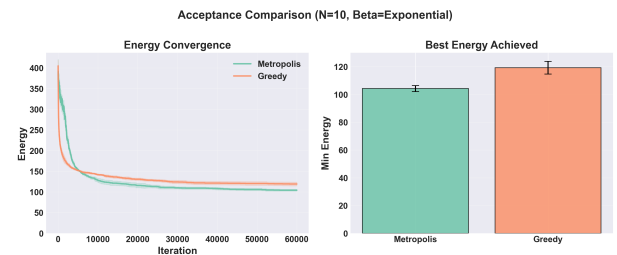
The MCMC chain satisfies three fundamental properties:

1. **Ergodicity** From any configuration, there exists a sequence of moves that reaches any other configuration. This ensures the chain can explore the entire state space.
2. **Aperiodicity** The probabilistic acceptance criterion prevents the chain from cycling periodically through states.
3. **Stationary Distribution** As $t \rightarrow \infty$, the distribution of s_t converges to $\pi(s)$. Thus, the chain asymptotically spends more time in low-energy regions.

Together, these properties guarantee that running the chain long enough allows us to find configurations close to the global optimum (energy zero) with high probability.

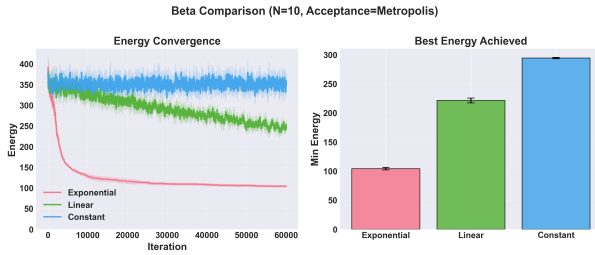
4 Experiments and results

4.1 Acceptance criterion



The performance of the Monte Carlo Markov Chain algorithm was evaluated using two acceptance criteria: the Metropolis-Hastings criterion and a Greedy acceptance rule. The Energy Convergence plot (left panel) displays the mean energy achieved over 60,000 iterations, averaged across four independent runs. Initially, the Greedy algorithm demonstrates a faster descent, achieving lower energy states more quickly. This rapid initial decrease is due to its pure exploitation strategy, immediately accepting any proposed move that leads to a lower energy. However, beyond this initial phase, the Metropolis-Hastings algorithm demonstrates superior long-term convergence. It gradually overtakes the Greedy approach, achieving a lower final energy state. This outcome highlights the crucial exploration-exploitation trade-off.

4.2 β scheduling

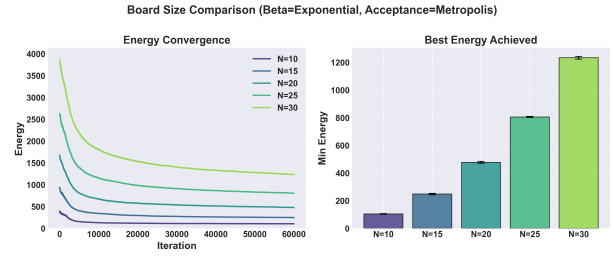


This experiment compared three β schedules in the Metropolis-Hastings algorithm: Constant, Linear, and Exponential (Simulated Annealing).

- The Energy Convergence plot shows that the Exponential β schedule achieved the fastest and deepest energy reduction. This rapid cooling schedule effectively balances initial exploration (low β) with final exploitation (high β), allowing the system to escape local minima.
- Linear β provided moderate improvement but converged slowly to a higher minimum.
- Constant β performed the worst, quickly becoming trapped at a high-energy state due to its persistent acceptance of high-energy moves.

The Best Energy Achieved chart confirms the ranking: Exponential β yielded the lowest minimum energy, confirming it is the superior method for this optimization problem due to its dynamic control of the acceptance probability, which mimics a physical annealing process.

4.3 Energies at different N



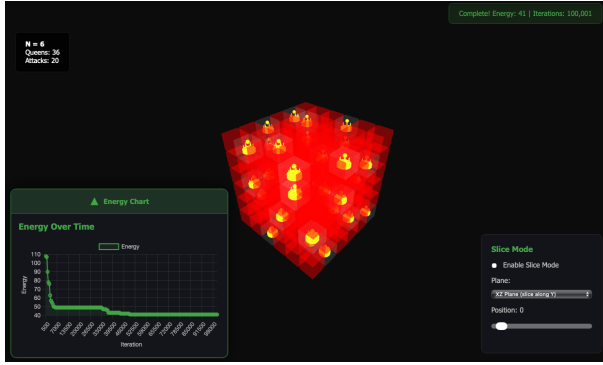
Two primary insights are drawn from the analysis of the Energy Convergence and Best Energy Achieved plots:

- **Computational Budget Requirement:** The larger the N , the more computational budget (i.e., iterations) is needed for convergence. As N increases, the energy curve's rate of descent decreases significantly and the total required length of the simulation to fully explore the search space grows. For instance, the energy curve for $N = 30$ is much shallower and still visibly decreasing at 60,000 iterations compared to $N = 10$.
- **Convergence Energy Scaling:** The larger the N , the higher is the convergence energy reached by the algorithm. The minimum energy achieved is directly correlated with N . This confirms the increased difficulty of locating the true global minimum in the more complex, higher-dimensional energy landscape. The algorithm, limited by 60,000 steps, becomes increasingly trapped in higher, sub-optimal local minima as N increases.

This scaling behavior is characteristic of NP-hard combinatorial optimization problems, where resource requirements grow rapidly with the problem size.

5 Visualization

To better understand the dynamics of the Markov chain beyond aggregate plots, we built an interactive web-based visualization of the solver. The interface renders the current 3D configuration of queens in real time and plots the energy trajectory as the chain progresses. This tool is especially useful to qualitatively inspect mixing behavior (e.g., how often queens move, whether the chain gets stuck) and to debug/compare parameter choices (temperature schedule, acceptance rule).



User interface of the visualization tool

How to run. From the repository root, run:

```
cd webapp
python app.py
```

Then open `http://127.0.0.1:5000` in a browser and click *Start MCMC Simulation* to set parameters and launch a run. During the simulation, the visualization emphasizes queen movements (attacked cells are de-emphasized) while the energy curve is updated live.

6 Conclusion

We formulated the 3D N^2 -queens problem as an energy minimization task and solved it using an MCMC Metropolis-Hastings sampler with simulated annealing. A key practical ingredient is the efficient energy bookkeeping that enables constant-time ΔE evaluation for single-queen moves, making long runs feasible. Empirically, we observed that annealing schedules (in particular exponential cooling) significantly improve convergence compared to constant-temperature baselines, while minimal achieved energy increases as N grows, reflecting the combinatorial hardness of the problem. Overall, the method provides a flexible, modular framework to explore stochastic optimization on this challenging 3D constraint landscape, and the accompanying interactive visualization helps interpret and communicate the solver's behavior during a run.